kashish chauhan 240251660

COMPUTER VISION LABSHEET

Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Aim: To implement and evaluate different image segmentation techniques for partitioning digital images into meaningful regions, helping object localization, scene understanding, and later computer vision tasks.

Source image used: Real-world photograph of coins from the standard *scikit-image* sample dataset. It is used only as an input image; the segmentation outputs shown in this report were actually generated by running the code.

Techniques implemented: Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering.



Figure 1. Real-world input image used for the experiment.

1. Purpose

Image segmentation divides an image into meaningful regions or objects. Instead of treating the complete image as one large collection of pixels, segmentation tries to isolate the important parts. This makes later tasks such as object detection, medical image analysis, navigation, and inspection easier.

In this experiment, five techniques are implemented and compared. Each technique makes segmentation decisions in a different way: thresholding uses intensity, adaptive thresholding uses local neighbourhood information, watershed uses region boundaries and markers, and K-Means groups pixels into clusters.

2. Prerequisites

a. Basic Digital Image Processing concepts.

b. Python programming knowledge.

c. OpenCV (cv2), NumPy, Matplotlib, Scikit-learn and scikit-image.

d. Understanding of grayscale images, histograms and clustering.

e. Jupyter Notebook, Google Colab or VS Code.

3. Experiment Steps

1. Import the required libraries and load a real-world image.
2. Convert/preprocess the image and reduce noise using Gaussian Blur.
3. Apply Global Thresholding to separate foreground and background.
4. Apply Otsu's Thresholding to automatically select a threshold.
5. Apply Adaptive Thresholding for local/uneven illumination.
6. Apply Watershed Segmentation to separate touching or overlapping objects.
7. Apply K-Means Clustering for pixel-based segmentation.
8. Compare the outputs using boundaries, segmentation quality and efficiency.
9. Visualize the results and identify a suitable technique.
1. Summarize observations and relate segmentation to advanced CV applications.

4. Humanized Pseudocode

```
START 1. Import OpenCV, NumPy, Matplotlib, scikit-image and scikit-learn. 2. Load a real-world image of coins. 3. Resize the image and apply Gaussian Blur to reduce small noise. 4. Choose a fixed threshold and create the Global Threshold image. 5. Let Otsu's method calculate the threshold automatically and create its segmented image. 6. Apply Adaptive Thresholding using a local neighbourhood. 7. Create foreground/background markers using thresholding and distance transform. 8. Run the Watershed algorithm and mark the boundaries of separated objects. 9. Convert pixels into a feature array and group them into 3 K-Means clusters. 10. Convert cluster labels back into image form. 11. Display all segmentation results. 12. Print important values such as Otsu threshold, marker count and cluster centers. END
```

4. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from sklearn.cluster import KMeans
```

```
# 1. Load a real-world image image
= data.coins()
image = cv2.resize(image, (640, 480))

# 2. Preprocess
gray = image
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# 3. Global Thresholding
T = 110
_, global_thresh = cv2.threshold(
    blur, T, 255, cv2.THRESH_BINARY
)

# 4. Otsu Thresholding
otsu_T, otsu_thresh = cv2.threshold(
    blur, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# 5. Adaptive Thresholding
adaptive_thresh = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    31, 5
)

# 6. Watershed Segmentation
_, binary = cv2.threshold(
    blur, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(
    binary, cv2.MORPH_OPEN, kernel, iterations=2
)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(
    dist, 0.32 * dist.max(), 255, 0
)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

num_markers, markers = cv2.connectedComponents(sure_fg) markers
= markers + 1
markers[unknown == 255] = 0

color = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
markers = cv2.watershed(color, markers)
color[markers == -1] = [0, 0, 255]

# 7. K-Means Clustering
pixels = gray.reshape((-1, 1)).astype(np.float32)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10) labels
= kmeans.fit_predict(pixels)
centers = kmeans.cluster_centers_.flatten() order
= np.argsort(centers)
rank = np.empty_like(order)
rank[order] = np.arange(3)
segmented = (
    rank[labels].reshape(gray.shape) * 127
).astype(np.uint8)

# 8. Visualize
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(gray, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap='gray')
plt.title('Global Threshold')
plt.axis('off')

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap='gray')
plt.title(f'Otsu (T={otsu_T:.1f})')
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap='gray')
plt.title('Adaptive Threshold')
plt.axis('off')

plt.subplot(2, 3, 5) plt.imshow(cv2.cvtColor(color,
cv2.COLOR_BGR2RGB))
```

```
plt.title('Watershed')
plt.axis('off')

plt.subplot(2, 3, 6)
plt.imshow(segmented, cmap='gray')
plt.title('K-Means (K=3)')
plt.axis('off')

plt.tight_layout()
plt.show()

print("Image shape:", gray.shape) print("Otsu
threshold:", otsu_T) print("Watershed
markers:", num_markers - 1) print("K-Means
centers:", np.sort(centers))
```

6. **Code Explanation**

Image loading: The experiment uses the real coins photograph from scikit-image. The image is resized to 640 × 480 so processing and visualization remain manageable.

Gaussian Blur: A 5×5 Gaussian filter smooths small intensity variations before thresholding. This helps reduce the effect of minor noise.

Global Thresholding: Every pixel is compared with a fixed value T = 110. Pixels on one side of the threshold are assigned to the foreground and the others to the background.

Otsu's Thresholding: Otsu examines the image histogram and automatically selects a threshold that separates two intensity groups as well as possible.

Adaptive Thresholding: Instead of one threshold for the complete image, a threshold is calculated from a local neighbourhood. This is useful when illumination is not uniform.

Watershed: Distance transform and connected components create markers. Watershed then grows regions and places boundaries where different regions meet, making it useful for touching objects.

K-Means: Each pixel intensity is treated as a one-dimensional feature. K-Means groups the pixels into three clusters according to intensity and maps those labels back to an image.

5. **Actual Output / Results**

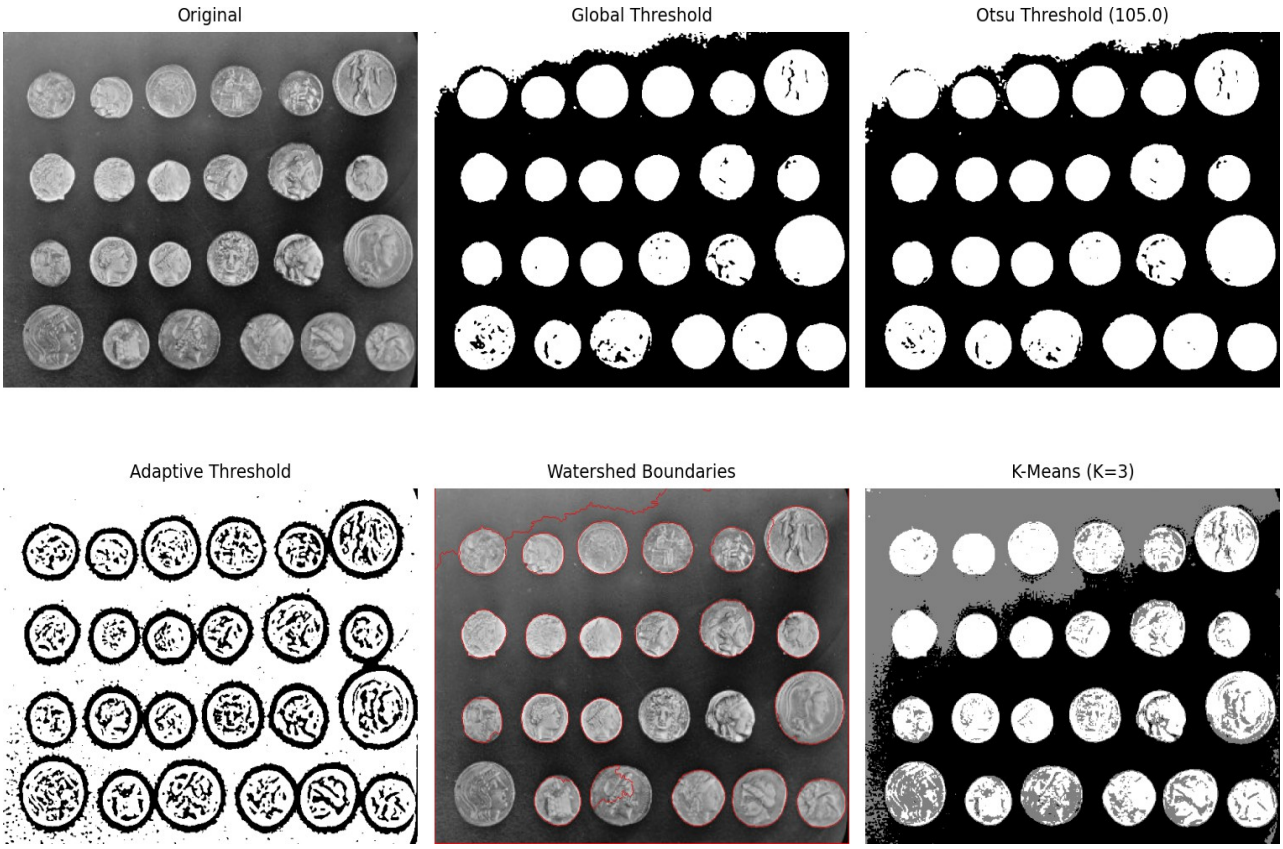


Figure 2. Actual segmentation outputs generated by executing the program.

Measured output	Actual result
Input image size	640 × 480 pixels
Global threshold	110
Otsu threshold	105.0
Global foreground pixels	38.32%
Otsu foreground pixels	40.52%
Adaptive foreground pixels	75.58%
Watershed object markers	23
K-Means clusters	3
Sorted K-Means centers	48.52, 104.90, 170.21

8. **Detailed Output Analysis**

Global Thresholding: It is simple and fast, but a single fixed threshold may not work equally well for every part of an image.

Otsu's Thresholding: It removes the need to manually select the threshold. For this image, the program selected the threshold automatically from the histogram.

Adaptive Thresholding: It handles local intensity changes better because the threshold changes according to the neighbourhood.

Watershed: The red boundaries in the output indicate region borders produced by the algorithm. It is especially useful when separate objects touch each other.

K-Means: Pixels are grouped into three intensity-based clusters. It provides a simple clustering-based segmentation, although it does not explicitly understand object shapes or semantic meaning.

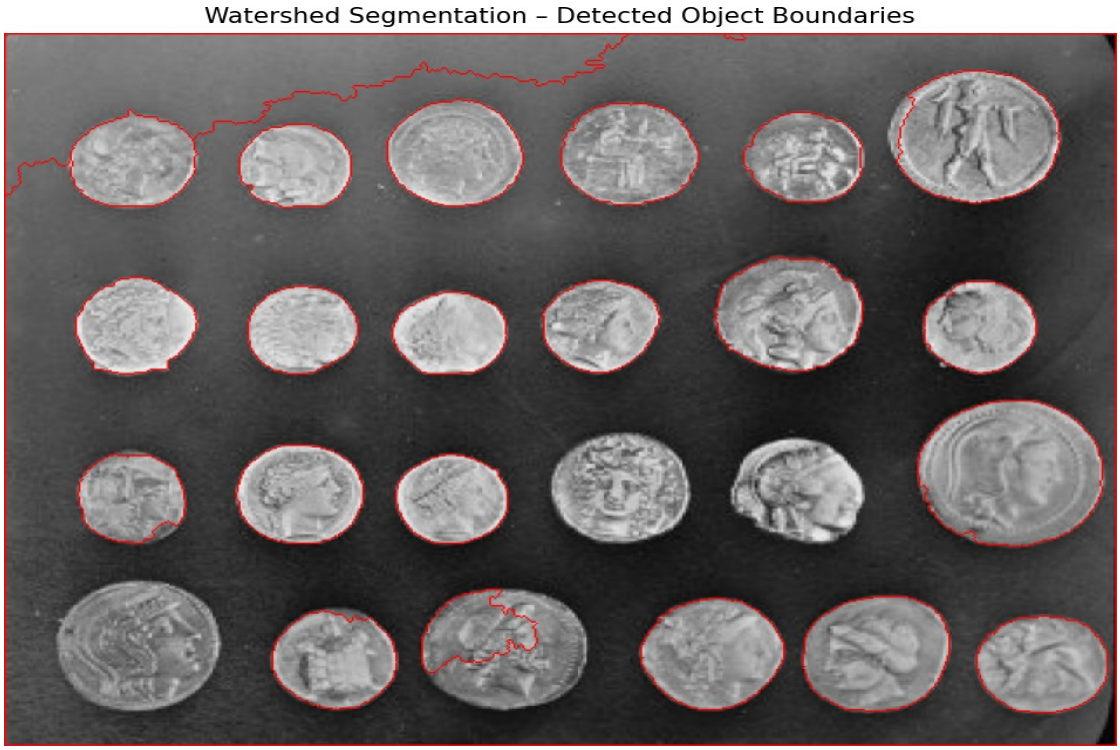


Figure 3. Watershed output with detected region boundaries.

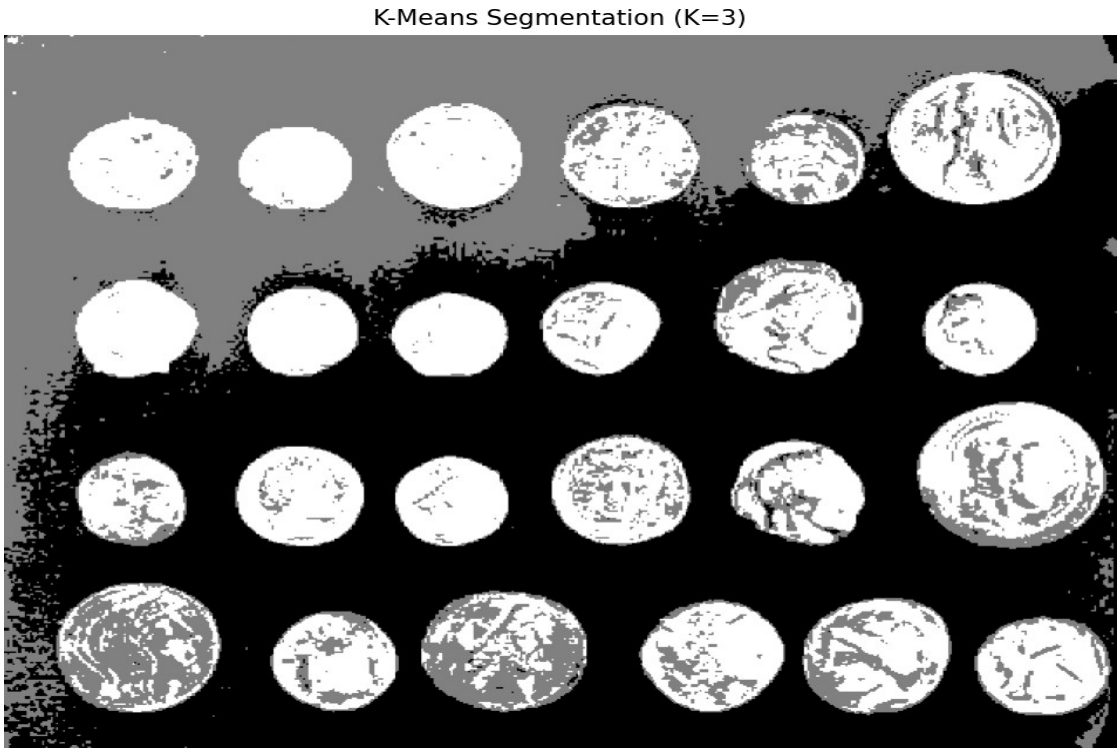


Figure 4. K-Means segmentation with K = 3.

9. **Comparison of Techniques**

Technique	Main idea	Strength	Limitation
Global Threshold	One fixed intensity threshold	Very simple and fast	Sensitive to illumination
Otsu	Automatically chooses threshold	No manual threshold selection	Best for reasonably bimodal histograms
Adaptive	Local thresholds	Works better with uneven illuminatio	nMore computation and parameter sensitivity

Technique	Main idea	Strength	Limitation
Watershed	Region growing from markers	Separates touching objects	Can over-segment without good markers
K-Means	Clusters pixels by features	Flexible and useful for intensity/color	gNreoeuqpsinKg and may ignore object boundaries

10. Questions and Answers

Q1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects. It is fundamental because it isolates the Region of Interest (ROI) from irrelevant background pixels. Once the important region is separated, tasks such as object detection, recognition, measurement and tracking become easier and more accurate. For example, in a medical image, segmentation can separate a suspected abnormal region from surrounding tissue.

Q2. Differentiate between Image Segmentation and Image Classification.

Image classification assigns a label to an entire image, such as 'cat', 'car' or 'normal'. Image segmentation works at the pixel/region level and determines which pixels belong to which region or object. Classification answers 'What is in the image?', while segmentation answers 'Which pixels belong to the object or region?'.

Q3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding uses one manually selected threshold for the complete image. Otsu's method automatically finds a threshold from the image histogram by trying to obtain good separation between intensity classes. Adaptive Thresholding calculates a threshold separately for local neighbourhoods, so it performs better when different parts of an image have different illumination levels.

Q4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Watershed is a region-based segmentation method inspired by the idea of water flowing over a landscape. Image intensity is treated like height and markers define starting regions. The algorithm grows regions until boundaries meet. These boundaries help separate objects that touch each other, which is why watershed is commonly used for touching cells, coins and other connected objects.

Q5. How is K-Means Clustering applied to image segmentation?

First, every pixel is represented as a feature vector. For grayscale segmentation, the pixel intensity can be used as the feature. K-Means selects K cluster centers, assigns each pixel to the nearest center, updates the centers, and repeats until the assignments become stable. The final cluster labels are reshaped into the original image size to create the segmented image.

Q6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-based methods use intensity rules to divide pixels into regions. They are usually simple and fast, but their performance can decrease when illumination changes or object and background intensities overlap. Clustering-based methods such as K-Means group pixels according to similarity in their features. They are more flexible, but the number of clusters and feature representation must be selected carefully.

Q7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Major challenges include noise, shadows, reflections, low contrast, overlapping objects, texture, similar foreground/background colors, and uneven illumination. A single threshold may fail in such conditions.

Preprocessing, adaptive methods, better markers, color spaces, clustering, or deep learning can be used to improve the result.

Q8. Mention five real-world applications where image segmentation plays a critical role.

1) Medical imaging for tumour or organ segmentation. 2) Autonomous vehicles for road, lane and object regions. 3) Satellite imagery for land-use and building analysis. 4) Industrial inspection for defect and component measurement. 5) Agriculture for crop, leaf and disease-region analysis.

Q9. How does image segmentation improve the performance of object detection and image recognition systems?

Segmentation separates useful object pixels from background pixels. This reduces irrelevant information and can provide more accurate object boundaries, size and shape. The cleaner region can then be used by detection or recognition algorithms, improving localization and sometimes reducing false detections. In complex scenes, segmentation also helps distinguish nearby objects.

Q10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Traditional methods such as thresholding, watershed and K-Means are generally easier to implement, require less training data, and can be computationally light. However, they often depend on image intensity, parameters and scene conditions. U-Net is a deep encoder-decoder architecture widely used for pixel-wise segmentation, especially in medical imaging. Mask R-CNN performs instance segmentation by detecting individual objects and predicting a mask for each object. Deep learning methods can learn complex visual patterns and usually perform better on difficult datasets, but they require training data, model computation and suitable hardware.

11. **Final Observations**

- Global thresholding is the simplest and fastest approach when foreground and background intensities are clearly separated.

f. Otsu's method is convenient because the threshold is selected automatically.

g. Adaptive thresholding is more suitable when illumination changes across the image.

- Watershed is particularly useful for separating touching objects, but marker selection is important to avoid over-segmentation.

h. K-Means provides a clustering-based alternative and can be extended to multi-channel/color features.

- No single segmentation technique is best for every image. The appropriate method depends on image content, illumination, noise, object overlap and the final application.

11. **Role in Advanced Computer Vision**

Segmentation is an important foundation for modern computer vision. It can provide accurate regions for object detection, tracking, recognition, measurement, medical analysis, autonomous navigation and industrial inspection. Traditional methods are useful when the scene is controlled and the image characteristics are known, while deep learning methods are valuable when complex patterns must be learned from large